

Course Project

A custom APB UART IP

You are required to design a custom UART IP that is wrapped with the AMBA APB interface that can be a part of an SoC.

Objective

The goal of this project is to design and understand a **Universal Asynchronous Receiver Transmitter (UART) peripheral wrapped with an AMBA APB interface**. Students will implement the APB slave logic that communicates with a UART core, providing register-based access for control, status, data, and baud rate configuration.

This project helps students learn:

- How to build a memory-mapped peripheral with APB.
- How to map registers to control UART hardware.
- How to handle write/read operations, status updates, and synchronization.
- How to integrate and test UART communication using a bus interface.

1. UART

UART stands for Universal Asynchronous Receiver/Transmitter.

It is a simple serial communication method where data is sent one bit at a time over a single wire in one direction, plus another wire for the reverse direction.

It's point-to-point, meaning one transmitter talks directly to one receiver. Many microcontrollers, FPGAs, and PCs have built-in UART hardware blocks because it's so widely used.

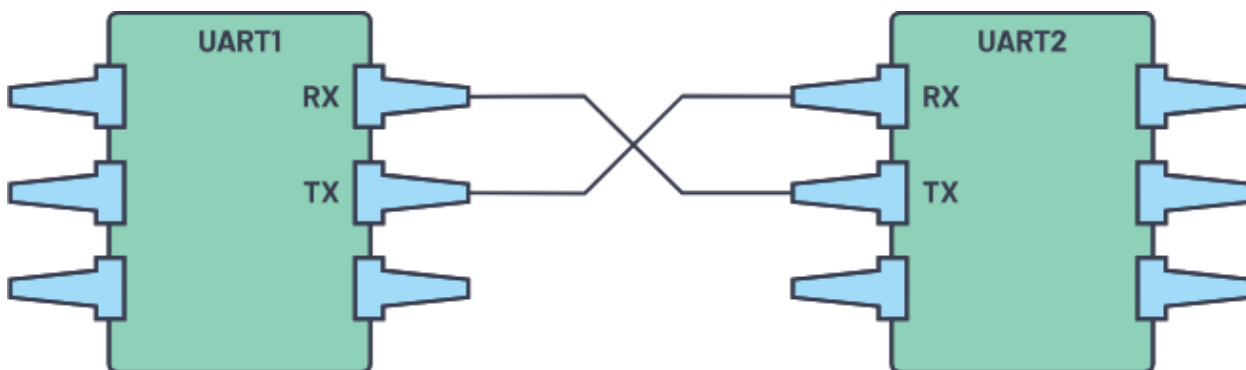


figure 1 UART chips communicating [source [ADI](#)]

UART uses 2 physical connections, one to transmit data & another to receive data.

Communication happens in **frames**. Each frame has a very specific structure, so the receiver knows where it starts, where it ends, and what data it contains.

The most common frame format is 8-N-1, which means:

8 data bits (sent least significant bit first)

N → No parity bit

1 stop bit

Before the data bits, there's a start bit (logic 0), and after them there's one or more stop bits (logic 1). The line stays idle (logic 1) between frames.

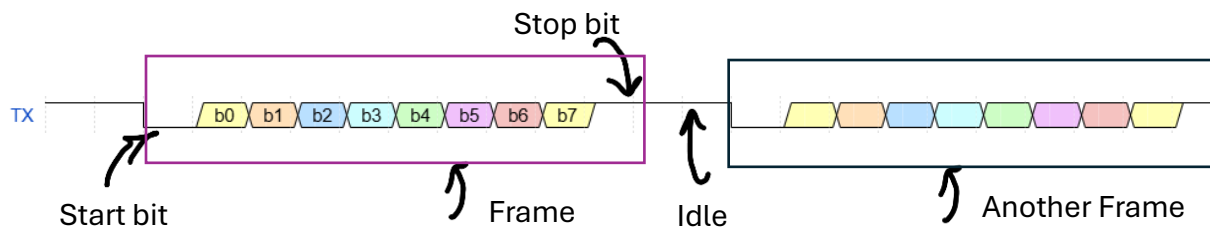


figure 2 UART frame

Before sending or receiving data, receiver and transmitter are configured to use the same **baud rate** (bits per second), e.g. **9600, 115200, etc.**

At 9600 baud → 1 bit period = $1/9600 \approx 104.167 \mu\text{s}$.

UART architecture

The receiver:

Only receives one byte at a time

- Waits for a falling edge (**start bit**).
- Samples the data bits **in the middle of each bit period**.
- Validates stop bit(s)
- Asserts **busy** when a byte is being received
- Asserts **done** when a byte is received

This is done through this suggested architecture

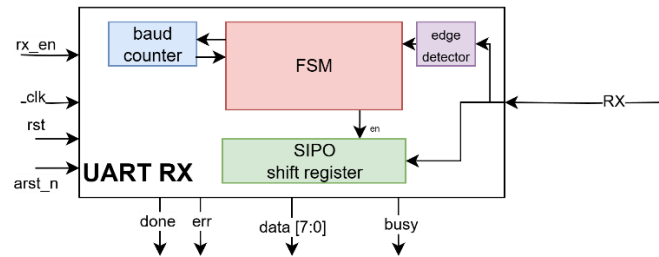


Figure 3 UART RX Architecture

Edge detector:

Catches the line transition from idle to low due to the start bit

Baud counter:

Baud counter is a down counting counter where the FSM controls its load value and gets a flag when it is zero

SIPO shift register

Enabled by the FSM, it samples the RX line every time it's enabled

RX FSM

- Loads the baud counter & waits until it reaches zero
- Control the SIPO shift register enable signal thus sampling bits in the middle
- Keeps track of how many bits have been sampled by the SIPO

Example State diagram

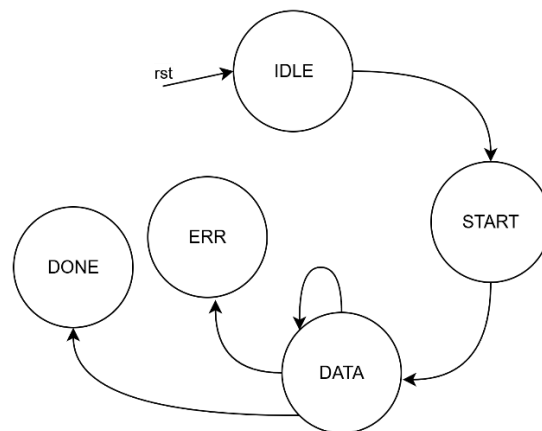


figure 4 RX FSM

- After starting condition, the FSM should wait for 1.5-bit time to enable the SIPO to sample the starting bit
- Then it should wait for 1 bit time
- The FSM should track the number of bits sampled & check for a correct stop bit

- It should assert **err** if framing error is detected

The Transmitter

- Sends only one byte at a time
- Appends start and stop bits
- Handles sending data with right bit times based on baud rate
- Asserts **done** after finishing sending the loaded byte
- Asserts **busy** while sending the loaded byte

Suggested Architecture

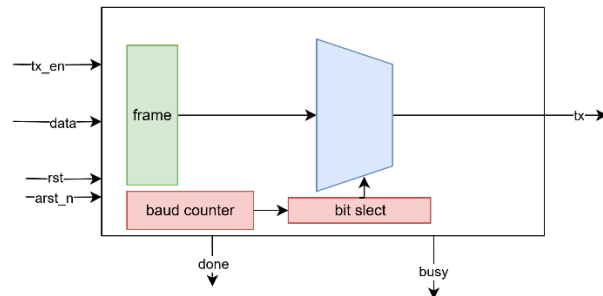


figure 5 UART TX

- **Baud counter** generates bit time based on required baud rate (9600) & system clock (50 MHz)
- **Bit select** keeps track of sent bits and holds the chosen bit on the line for the required bit time